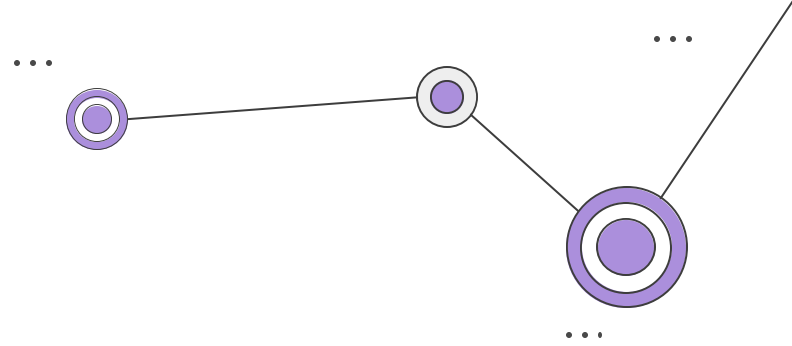
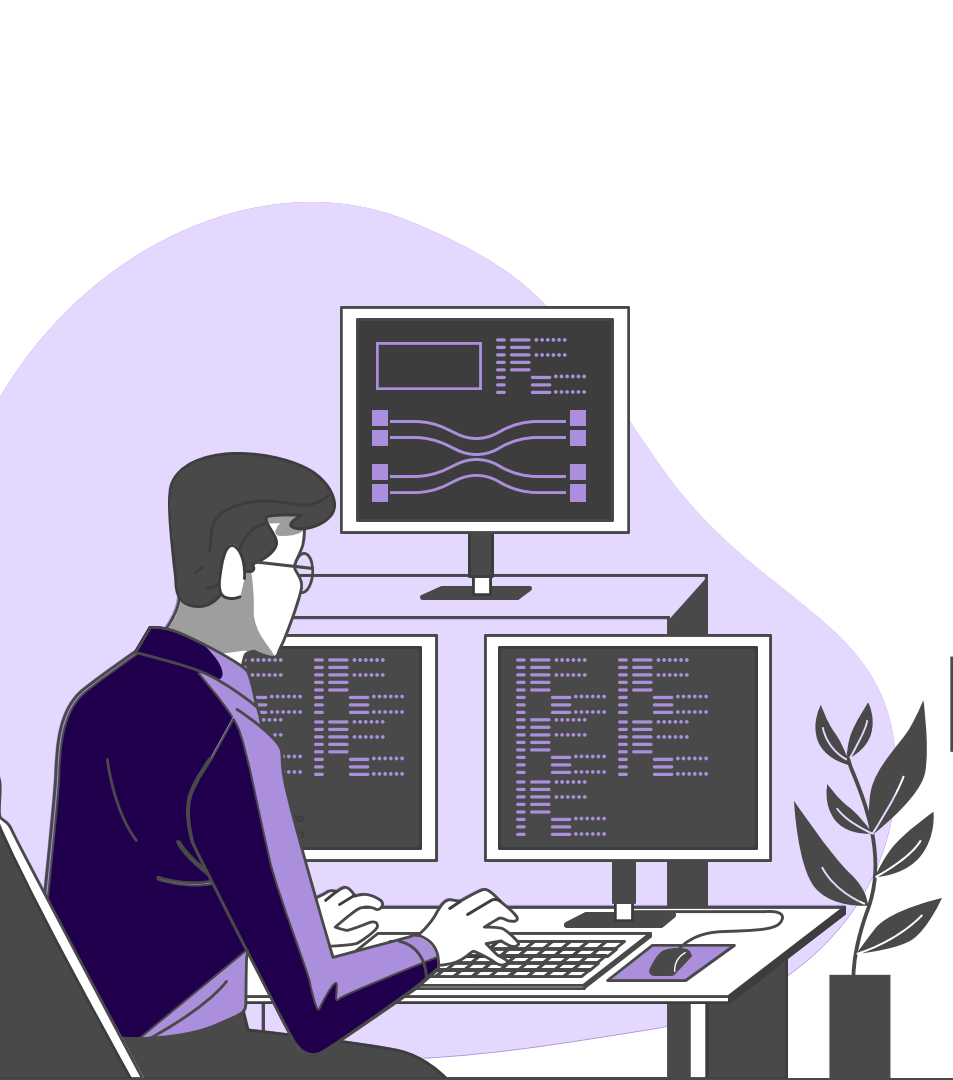
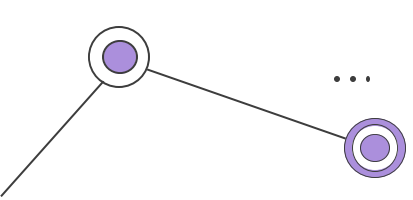


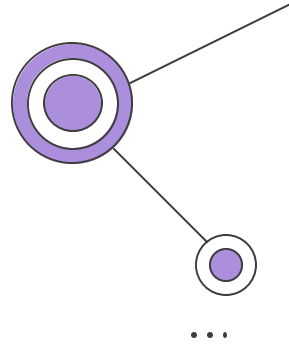


Ayudantía 3: Diagramación UML

Constanza Huerta
Martín Eyzaguirre



¿Por qué ver UML?



01

Estándar en la industria

02

Bueno para ver y mostrar soluciones rápidamente

03

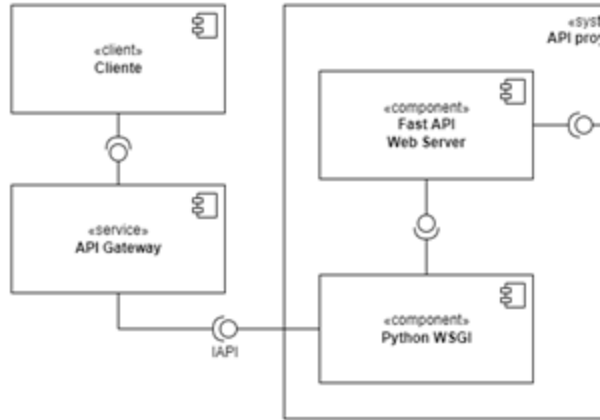
Refleja las ideas más fielmente

04

Se logra un mayor detalle de una implementación

Diagrama importante para este curso

Component Diagram (estructura)



Conectores (Cómo interactúan)
Componentes (Cajas y sus subcajas)
Interfaces (C requiere y O provee)

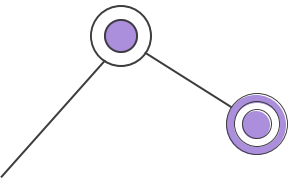
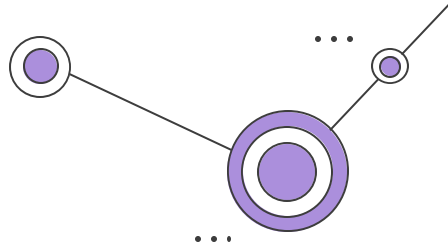
Componentes

Estereotipo, indica qué rol específico cumple el componente dentro del modelo. Usar simplemente <<component>> indica que es un componente de software autónomo y reutilizable. Otros estereotipos comunes son: «subsystem», «process», «service», «specification», «realization», «implement»

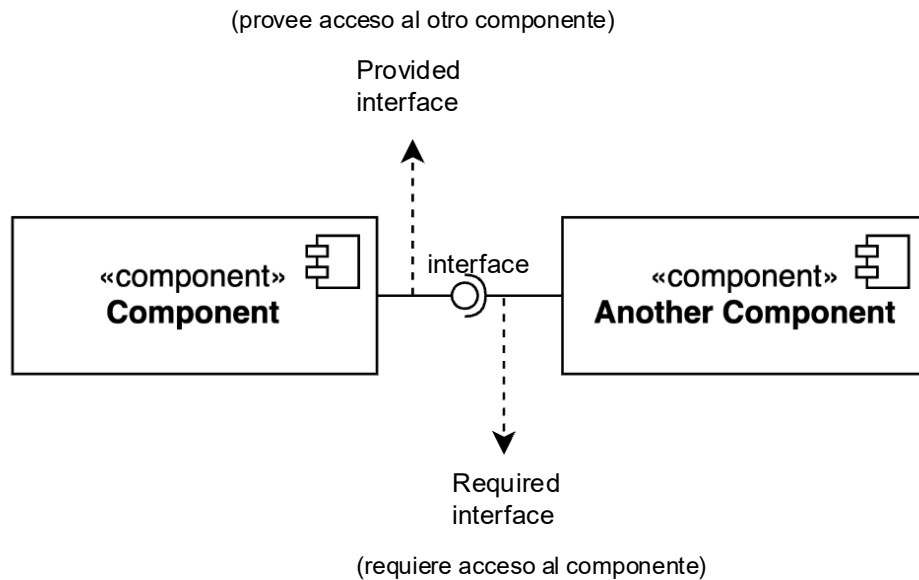


Icono que simboliza a un componente UML

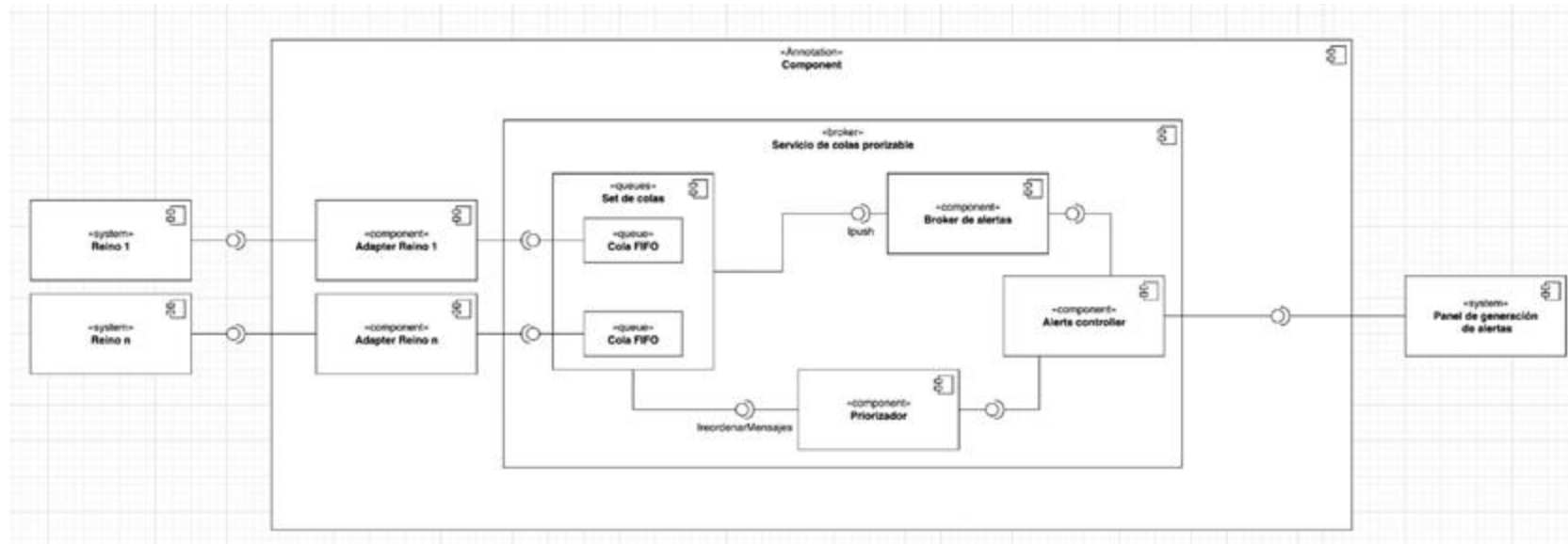
Nombre del componente, debe ser significativo y descriptivo. Usar sustantivos y no verbos



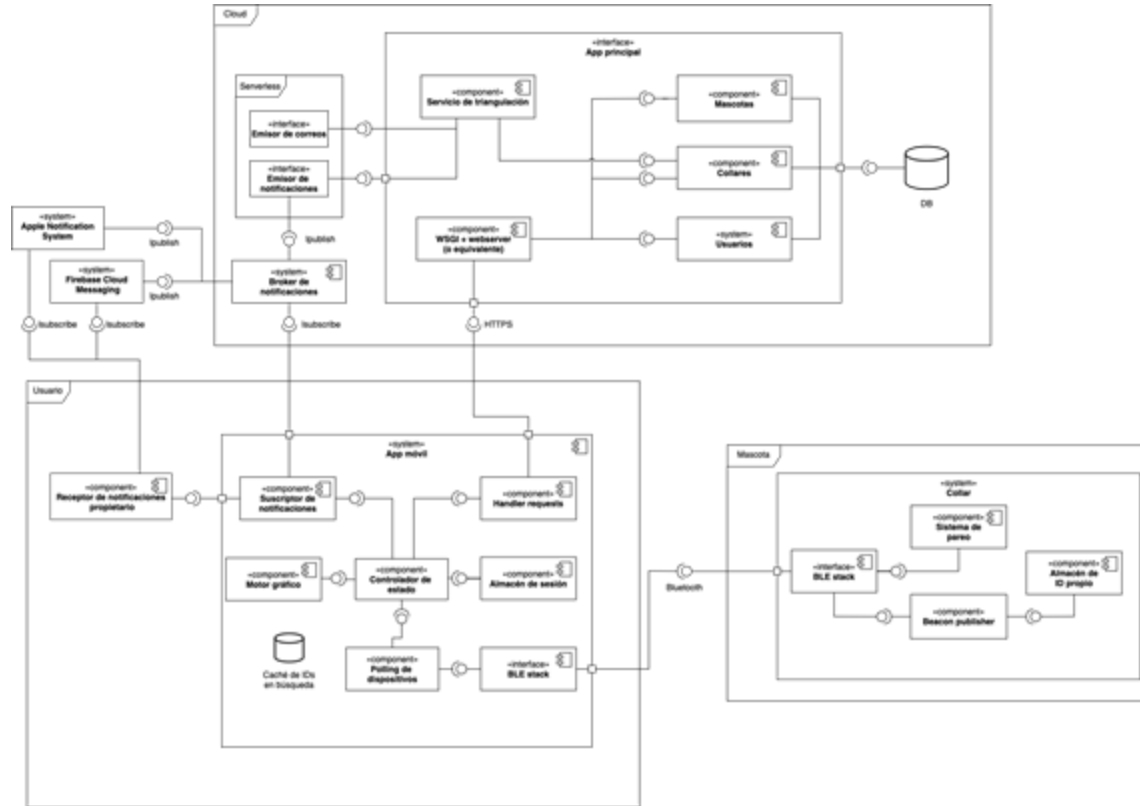
Interfaces



¿Qué se espera?

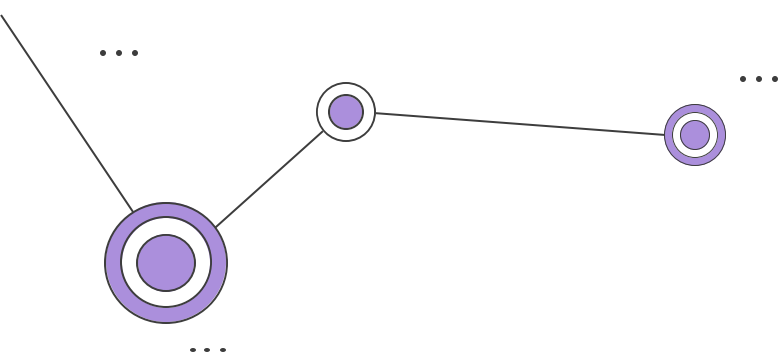


Todo dependerá del problema



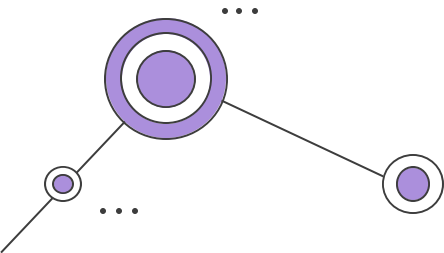
El detalle esperado depende de la complejidad del problema





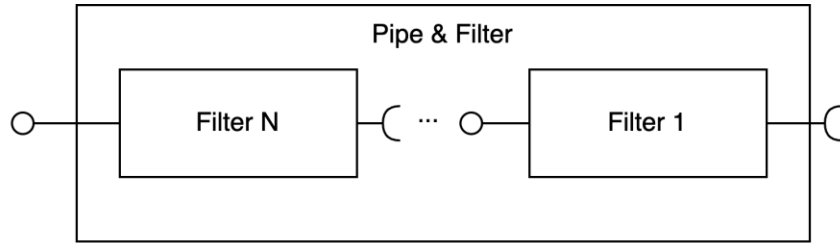
Aspectos clave al momento de diagramar

- ☐ Incluir los atributos de calidad (NFR) fundamentales del problema
- ☐ Incluir los estilos arquitectónicos identificados para solucionar el problema
- ☐ Usar interfaces y nombrarlas
- ☐ Incluir los requisitos funcionales
- ☐ No dejar todo general, incluir detalle cuando corresponda
- ☐ Conectar todos los componentes

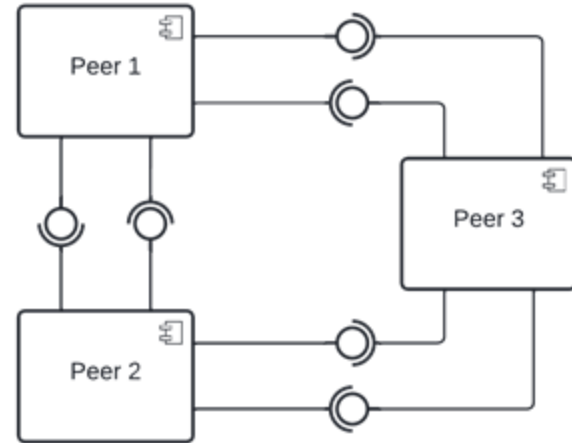


¿Cómo se ven algunos estilos típicos?

Pipe & Filter

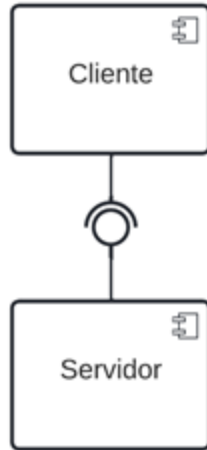


Peer To Peer



¿Cómo se ven algunos estilos típicos?

Cliente servidor

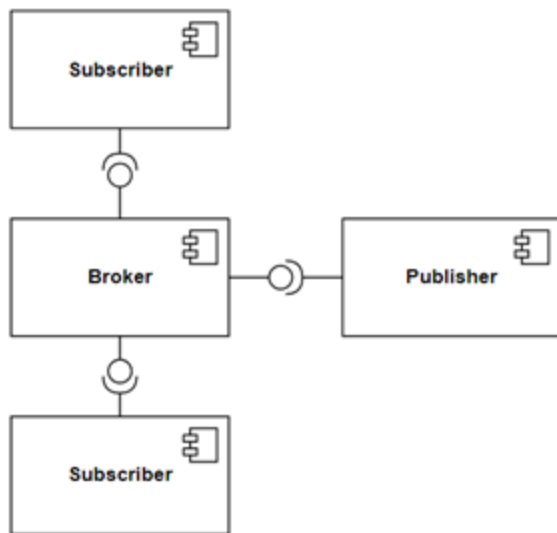


En capas

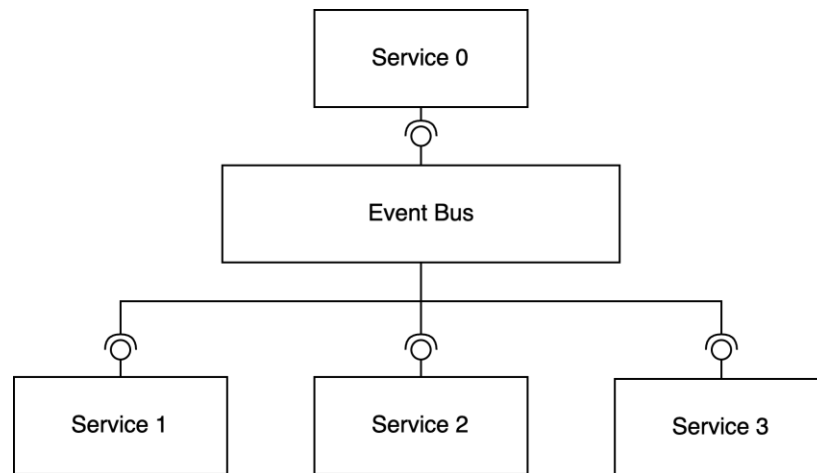


¿Cómo se ven algunos estilos típicos?

PubSub



Basado en eventos



Métodos clave



Inductivo: detalle a lo general

Tengo que armar un chat en tiempo real: debe aplicar invocación implícita

Tengo que almacenar fotos: hay que introducir un tipo de storage

...



Deductivo: general al detalle

Debo cubrir 1000 usuarios por hora: un balanceador de carga

Debo tener una arquitectura orientada a servicios (SoA)

...

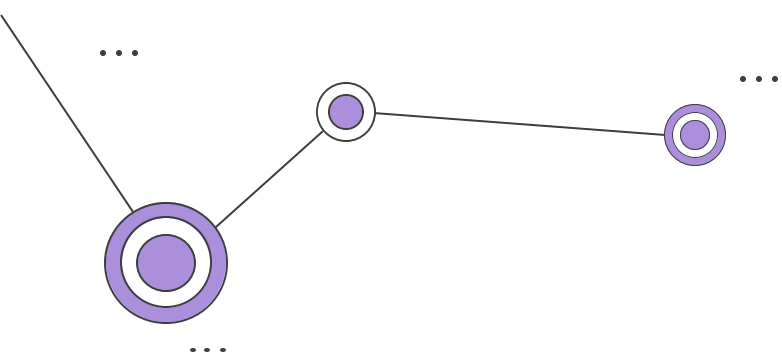


Referencia

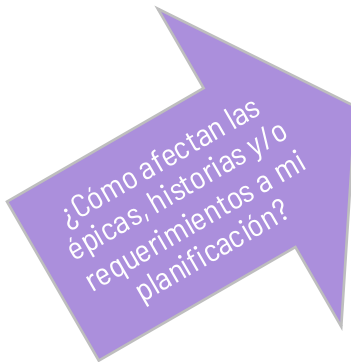
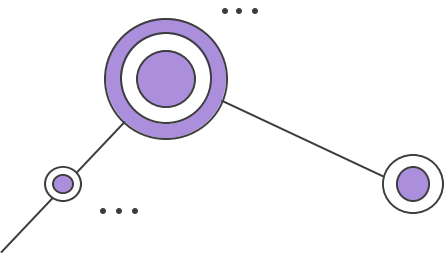
He visto esto en otro lado...

Hay componentes reusables parecidos

...



Preguntas Clave



Objetos que me permitan lograr
la comprensión del problema

Contextualización

Core

Especificaciones

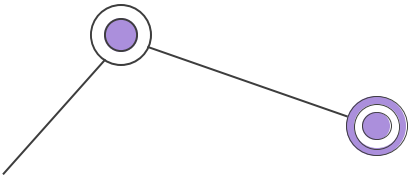
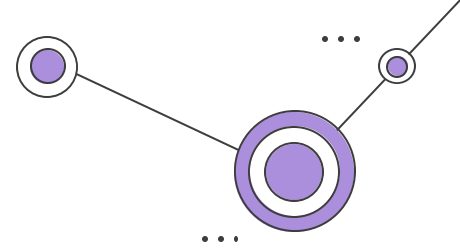
Detalles

Supuestos

Limitaciones

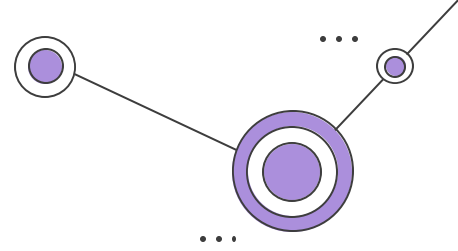


¿Qué buscar?

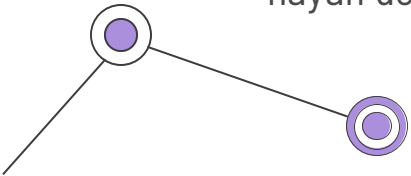


Errores clásicos

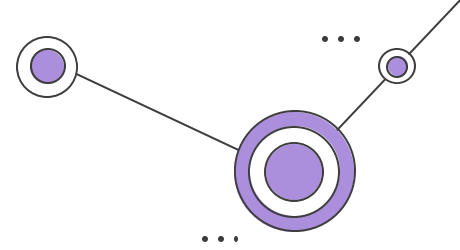
- Diagramas incompletos: falla en diagramar
- Componentes desagregados: no quedan claras las dependencias
- Conexiones inútiles: porque están ahí?
- No nombrar interfaces clave
- No incorporar estilos que se hayan declarado: inconsistente



¿Qué no hacer?



Ejemplo rápido



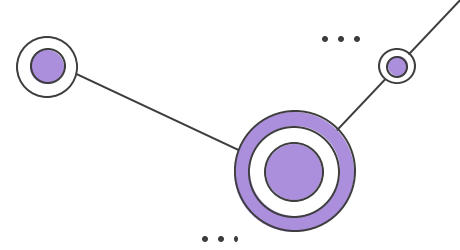
Ustedes y sus conocidos descubrieron que la empresa LegitBusiness lanzó un concurso para conseguir la mejor solución para el desarrollo de su nueva página web, la cual conectará a varias APIs de bancos con datos similares al parseo que ya tiene su aplicación, que se deberán procesar y filtrar para ser mostradas. Viendo esto ustedes decidieron participar por la experiencia y un chilión de dolares como premio.

Es así como con sus compañer@s deciden diseñar una aplicación con un frontend y backend separado, que se conecte al servicio disponible de cada banco y que procese y filtre la información para mostrarla en su página principal dentro de un Dashboard.

Como están cortos de tiempo, deciden hacer una implementación rápida en UML que les puede hacer ganar el concurso.

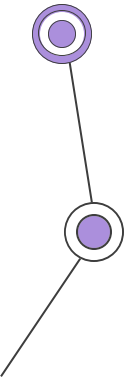


Ejemplo rápido

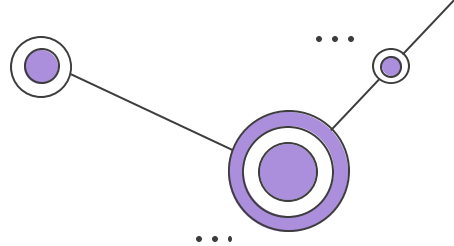


¿Cómo afrontamos el diseño? Pensemos solo en lo funcional mientras:

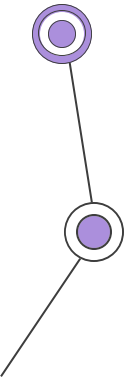
1. ¿Qué RNF creen que son necesarios priorizar en este problema? (al menos 2)
2. ¿Qué estilos de arquitectura creen que aplican de mejor manera como solución al problema? (al menos 2)
3. Diseñar el UML con lo decidido



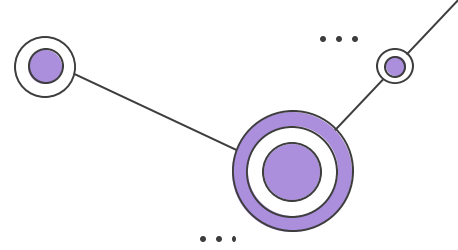
Ejemplo rápido



Pregunta extra: ¿Qué habría que hacer si no pudiéramos asegurar que los datos son similares a cómo los trabajamos en la aplicación?



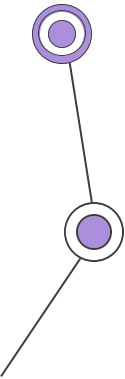
Ejemplo rápido



Ustedes y sus conocidos descubrieron que la empresa LegitBusiness lanzó un concurso para conseguir la mejor solución para el desarrollo de su nueva página web, **la cual conectará a varias APIs de bancos** con datos similares al parseo que ya tiene su aplicación, **que se deberán procesar y filtrar** para ser mostradas. Viendo esto ustedes decidieron participar por la experiencia y un chilión de dolares como premio.

Es así como con sus compañer@s deciden diseñar una aplicación con un **frontend** y backend separado, que se conecte al servicio disponible de cada banco y que procese y filtre la información para mostrarla en su página principal dentro de un **Dashboard**.

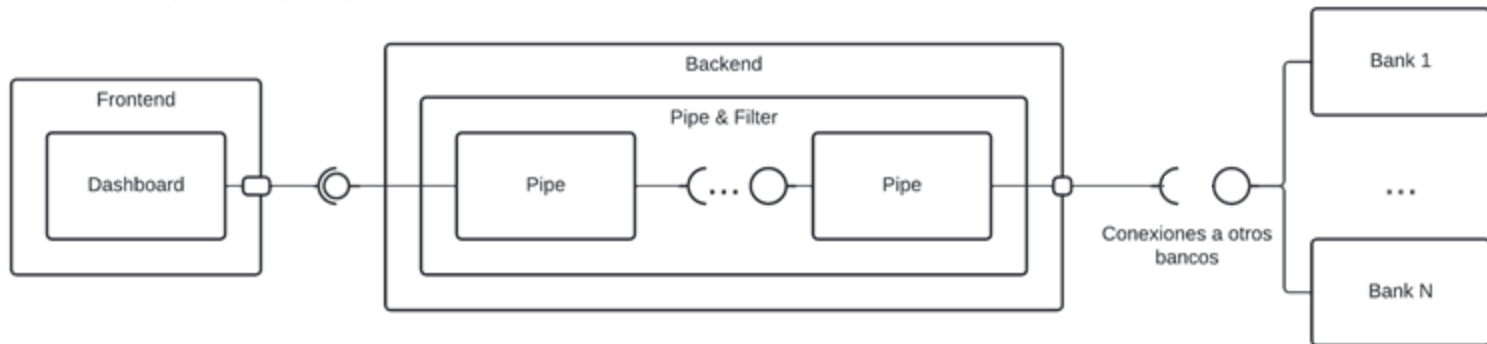
Como están cortos de tiempo, deciden hacer una implementación rápida en UML que les puede hacer ganar el concurso.



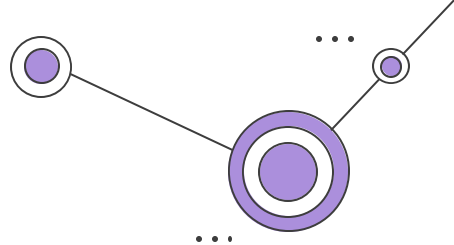
Solución Ejemplo rápido

1) Cliente/Servidor y Pipe & Filter

Pregunta extra: Habría que crear un componente adaptador para cada banco

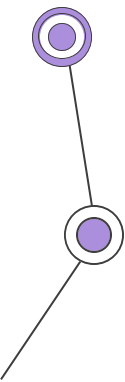


Pregunta I1 Anterior

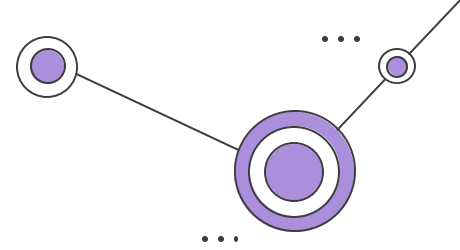


Las empresas salmoneras dentro de Chile tienen campos de cultivos para criar peces destinados al consumo, pero debido al crecimiento constante de estos campos, aumenta el riesgo de enfrentar problemas graves como enfermedades, escapes y otros. Si este crecimiento se vuelve desmesurado y no sustentable, puede provocar incluso la aparición de activistas medioambientales. Desde este problema, la empresa consultora LegitFoods en colaboración de su asociación medioambiental LegitGreen, los contrata para comenzar con los diseños de un proyecto inicial para controlar estas situaciones de sobrecrecimiento.

Los centros de cultivo pueden ser fuentes iniciales para distintos problemas al ecosistema como indicadores de riesgos para las regiones, escapes de pescados, intoxicaciones por virus en la zona, etc. Por lo mismo, se les propone **modelar un sistema de monitoreo general de estos centros para detectar esos problemas a tiempo y solucionarlos o mitigarlos**

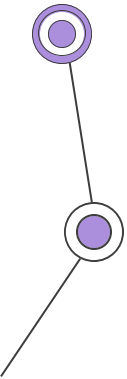


Pregunta I1 Anterior



Para cada cultivo, se instalarán **múltiples boyas inteligente** con distintos sensores que podrán **intercambiarse fácilmente** dependiendo de lo que se quiera medir controlados por sus respectivos *softwares*. Estos **sensores podrán medir calidad del agua, estado de los peces, nivel de algas, cuanto personal está atendiendo a los peces**, etc. Toda esta información recolectada se enviará, mediante una conexión directa vía **un gateway LORA**, a un ordenador ubicado en cada centro de cultivo.

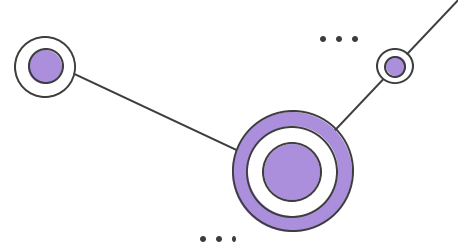
El computador **recibirá los datos continuamente** para que sean procesados por un componente **denominado FisherAnalyzer**, el cual **provee una interfaz** para recibir los datos, y requiere una interfaz para saber dónde enviar los datos analizados, tal como se muestra en la imagen adjunta.



Pregunta I1 Anterior

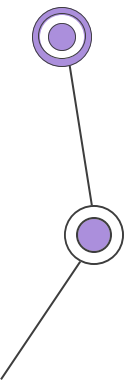


Pregunta I1 Anterior

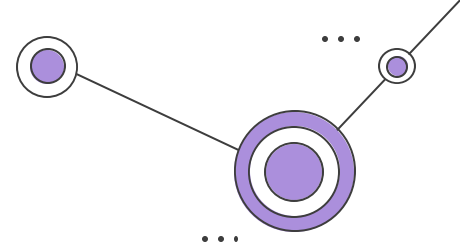


La información **generada será guardada localmente** para mostrarlos en **un dashboard** dentro del computador del centro de cultivo. Además, serán **enviados via satélite** a una central de LegitGreen, con capacidad de recibir y almacenar estos datos. Esta central permitiría conexiones con los sistemas internos y ERPs de distintas salmoneras y entidades regulatorias para que le soliciten información de diverso tipo.

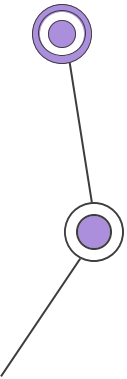
También tendrá un sistema capaz de permitir que distintas entidades se **conecten a ellos para obtener alertas sobre peligros inminentes o en curso**. Cada entidad (gobierno, civiles, otros) podría **registrarse para recibir mensajes o dejar un endpoint** abierto para ser llamado por el sistema principal.



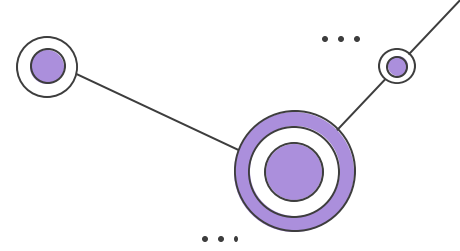
Pregunta I1 Anterior



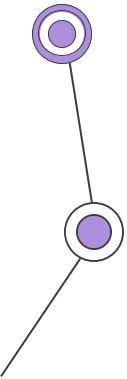
Indique (mínimo 3, pero pueden ser más) atributos de calidad vistos (RNF) en clases que Ud. considere son fundamentales en este problema. Enumérellos en orden de prioridad y justifique brevemente.



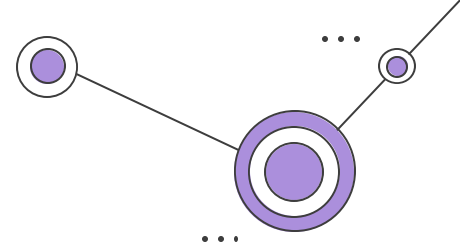
Pregunta I1 Anterior



Indique (mínimo 3, pero pueden ser más) atributos de calidad vistos (RNF) en clases que Ud. considere son fundamentales en este problema. Enumérellos en orden de prioridad y justifique brevemente.

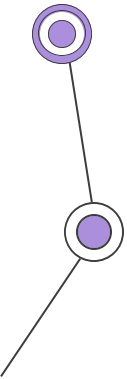


Pregunta I1 Anterior

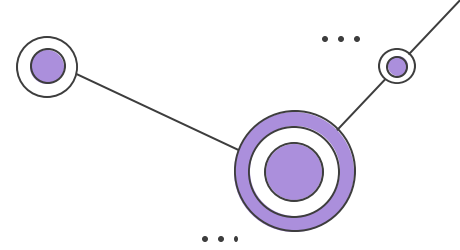


Indique (mínimo 3, pero pueden ser más) atributos de calidad vistos en clases que Ud. considere son fundamentales en este problema. Enumérelos en orden de prioridad y justifique brevemente.

- **Performance:** Debe ser rápido para responder, al estar recibiendo información continuamente y actualizándola.

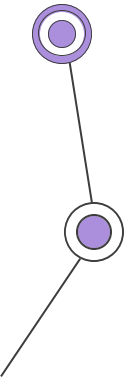


Pregunta I1 Anterior

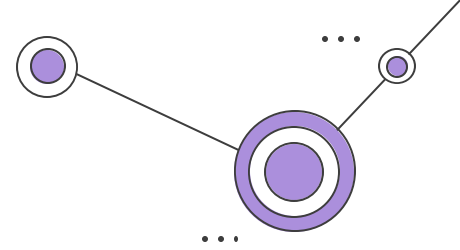


Indique (mínimo 3, pero pueden ser más) atributos de calidad vistos en clases que Ud. considere son fundamentales en este problema. Enumérellos en orden de prioridad y justifique brevemente.

- **Performance:** Debe ser rápido para responder, al estar recibiendo información continuamente y actualizándola.
- **Integrabilidad:** El sistema se debe integrar a otros entes externos de distintas maneras. Como se dice en el enunciado, con los ERP y los sistemas internos.

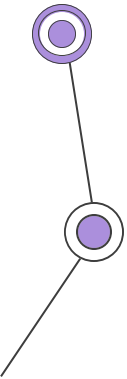


Pregunta I1 Anterior

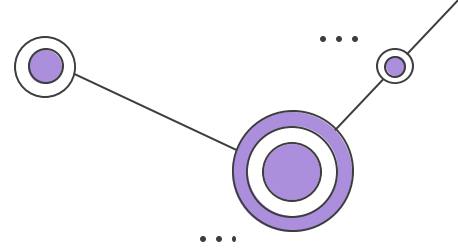


Indique (mínimo 3, pero pueden ser más) atributos de calidad vistos en clases que Ud. considere son fundamentales en este problema. Enumérelos en orden de prioridad y justifique brevemente.

- **Performance:** Debe ser rápido para responder, al estar recibiendo información continuamente y actualizándola.
- **Integrabilidad:** El sistema se debe integrar a otros entes externos de distintas maneras. Como se dice en el enunciado, con los ERP y los sistemas internos.
- **Seguridad:** No se pueden filtrar datos de la empresa que no sean públicos, como usuarios o datos gubernamentales que no están abierto al publico general.

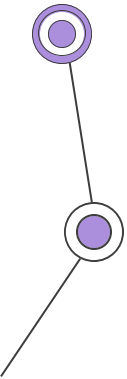


Pregunta I1 Anterior

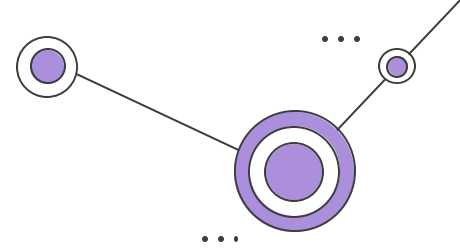


Indique (mínimo 3, pero pueden ser más) atributos de calidad vistos en clases que Ud. considere son fundamentales en este problema. Enumérelos en orden de prioridad y justifique brevemente.

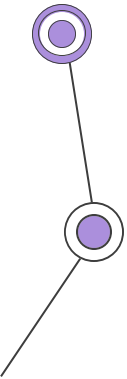
- **Performance:** Debe ser rápido para responder, al estar recibiendo información continuamente y actualizándola.
- **Integrabilidad:** El sistema se debe integrar a otros entes externos de distintas maneras. Como se dice en el enunciado, con los ERP y los sistemas internos.
- **Seguridad:** No se pueden filtrar datos de la empresa que no sean públicos, como usuarios o datos gubernamentales que no están abierto al publico general.
- **Confiableidad:** Los datos entregados deben ser reales, y los que consuman nuestro producto deben poder confiar en estos datos.



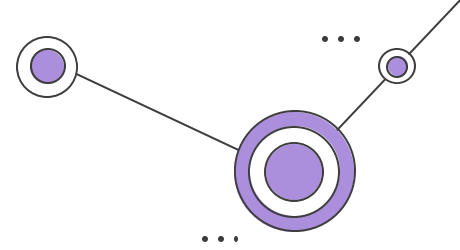
Pregunta I1 Anterior



Indique (mínimo 2, pero pueden ser más) los estilos arquitectónicos vistos en clases que va a emplear para solucionar el problema. Indique qué aspecto del problema ataca cada estilo (responsabilidad).

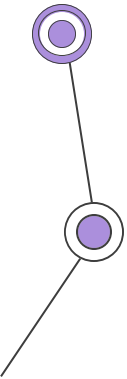


Pregunta I1 Anterior

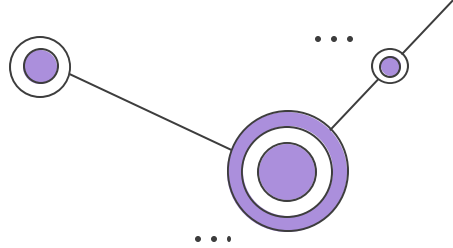


Indique (mínimo 2, pero pueden ser más) los estilos arquitectónicos vistos en clases que va a emplear para solucionar el problema. Indique qué aspecto del problema ataca cada estilo (responsabilidad).

- **Basado en Evento:** En el enunciado generamos alertas a distintas entidades, con los posibles peligros. Por esto mismo debemos generar un sistema de alerta que puede describirse como un estilo basado en evento. **También podría describirse como Pub/sub), pero dependería de como quieren diagramarlo y armar su solución.**

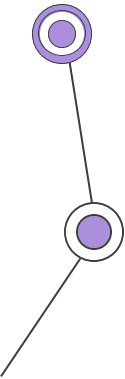


Pregunta I1 Anterior

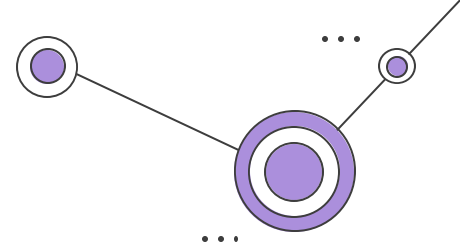


Indique (mínimo 2, pero pueden ser más) los estilos arquitectónicos vistos en clases que va a emplear para solucionar el problema. Indique qué aspecto del problema ataca cada estilo (responsabilidad).

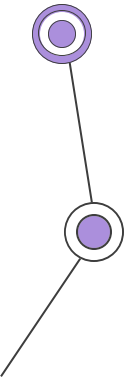
- **Basado en Evento:** En el enunciado generamos alertas a distintas entidades, con los posibles peligros. Por esto mismo debemos generar un sistema de alerta que puede describirse como un estilo basado en evento. **También podría describirse como Pub/sub), pero dependería de como quieren diagramarlo y armar su solución.**
- **MicroKernel:** Por como se describe el problema, podríamos diagramar un core central que maneja la información que sería Fisher analyze, y varios “plugins” alrededor que le entreguen la información. Además, este luego entregarla a otros sistema a través de los “plugins”.



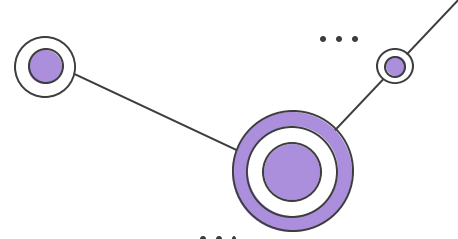
Pregunta I1 Anterior



Confeccione un diagrama de componentes UML indicando los componentes arquitectónicos y sus relaciones, además de sus estilos especificados previamente, tal que reflejen una solución al problema planteado. Añada tanto detalle como haga falta para que quede claro cómo soluciona el problema planteado. Incluya anotaciones explicando funcionamiento y supuestos, así como nombres de interfaces

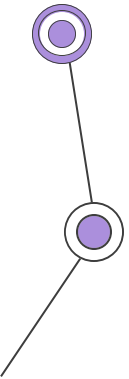


Pregunta I1 Anterior

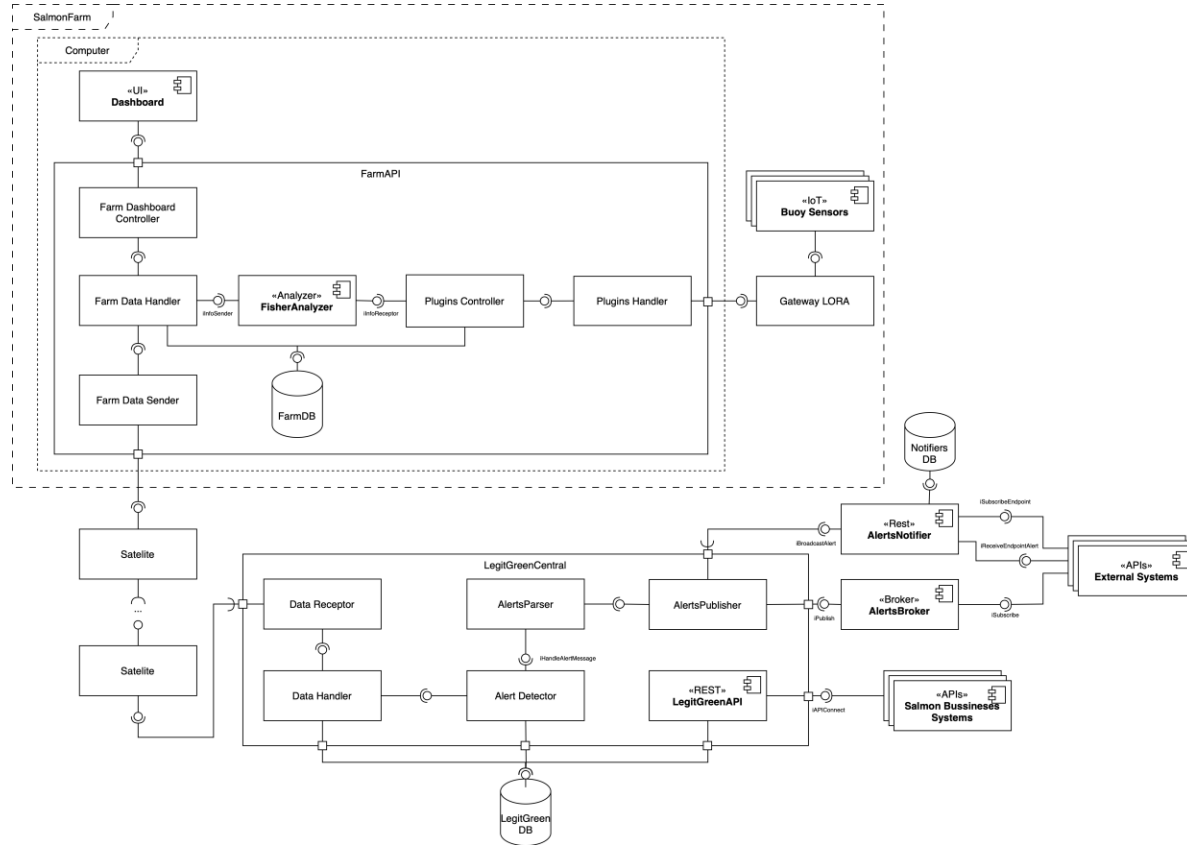


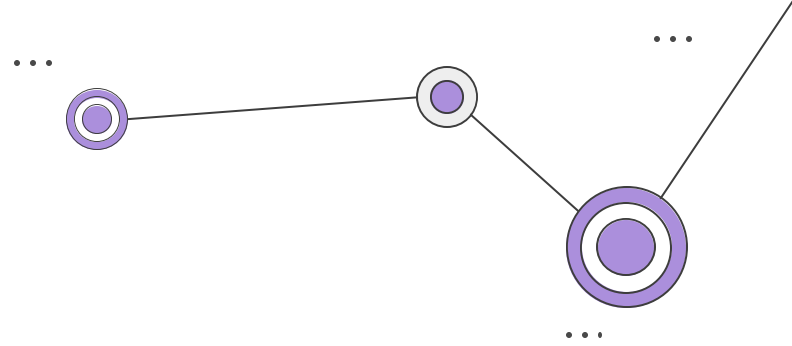
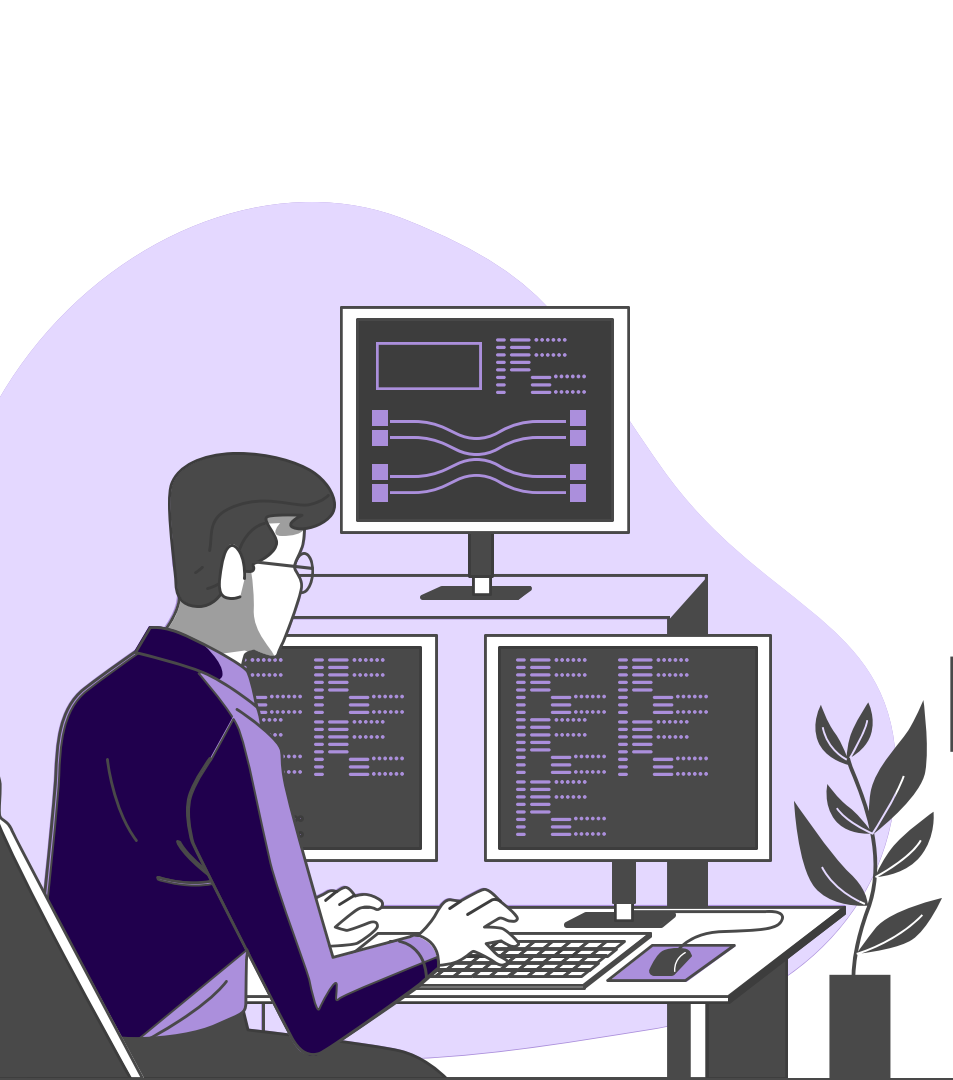
Identifique 2 falencias de su diagrama y comente como las solucionaría y las implicancias de estas soluciones

- Microkernel, tiene un punto único de falla. Esto se podría solucionar a través de replicar esta instancia.
- Debido a estar ocupando el estilo EDA, puede tener fuertes problemas de seguridad y falta de manejo de que información se envía. Por esta razón, se podría crear un confirmador de la información como capa intermedia antes de mandar las alertas.



Solución UML





Ayudantía 3: Diagramación UML

Constanza Huerta
Martín Eyzaguirre

Anexo: ¿Dónde hacerlos en el computador?

<https://app.diagrams.net>

<https://www.lucidchart.com/>

